## 第4章 数组



4.1 数组的定义

4.2 数组的顺序存储

4.3 矩阵的压缩存储

4.4 数组的链式存储结构

## 4.1 数组的定义



二维数组 $A_{mn}=\begin{pmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \cdots & a_{2n} \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ a_{m1} & a_{m2} & a_{m3} & \cdots & a_{mn} \end{pmatrix}$

可以看成一个线性表$A=(a_1\ a_2\ a_3\ \cdots\ a_p)$

当p=m时，有

$A=((a_{11}\ a_{12}\ a_{13}\cdots a_{1n}),(a_{21}\ a_{22}\ a_{23}\cdots a_{2n}),\ldots,(a_{m1}\ a_{m2}\ a_{m3}\cdots a_{mn}))$

当p=n时，有 每一元素由一行元素组成

$A=\left(\begin{pmatrix} a_{11} \\ a_{21} \\ \cdots \\ a_{m1} \end{pmatrix}\begin{pmatrix} a_{12} \\ a_{22} \\ \cdots \\ a_{m2} \end{pmatrix}\cdots\begin{pmatrix} a_{1n} \\ a_{2n} \\ \cdots \\ a_{mn} \end{pmatrix}\right)$ 每一元素是由一列元素组成

## 数组的定义(续)



结论：二维数组是一个线性表，其每一数据元素是一个一维数组。

以此类推，

三维数组是一个线性表，其每一数据元素是一个二维数组。

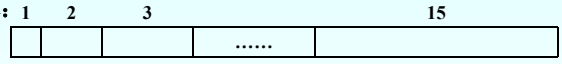
n维数组是一个线性表，其每一数据元素是一个n-1维数组。

## 4.2 数组的顺序存储



例:一维数组P[1:k],其第15个数据元素的存储终地址是1164H。如果 第i个数据占i个字节，求第1个数据元素的存储起始地址。

解：




15个数据元素占有的总存储为

$1+2+3+\cdots\cdots+15=120=78^H$

第一个数据元素的存储起始地址为

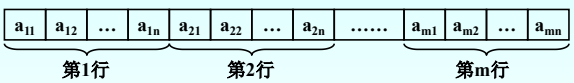
$1164^H-78^H+1=10ED^H$

数组是多维的结构 ⟷ 映射？ ⟷ 计算机存储单元是一维的结构

## 1. 二维数组$A_{mn}$的顺序存储结构



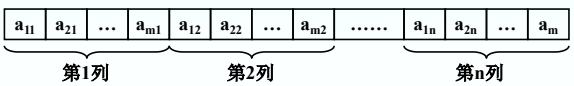
(1)按行优先的顺序存放




$Loc(a_{ij})=Loc(a_{11})+[(i-1)*n+(j-1)]*L$

$(1\leqslant i\leqslant m,\ 1\leqslant j\leqslant n)$

(2)按列优先的顺序存放




$Loc(a_{ij})=Loc(a_{11})+[(j-1)*m+(i-1)]*L$

$(1\leqslant i\leqslant m,\ 1\leqslant j\leqslant n)$

## 二维数组的例题（1）



例:二维数组：$1\leqslant i\leqslant 6,\ 1\leqslant j\leqslant 8$，则按行存储$a_{24}$和按列存储$a_{62}$地址相同。

解：按行 $Loc(a_{24})=Loc(a_{11})+[(i-1)*n+(j-1)]*L$

$=Loc(a_{11})+[(2-1)*8+(4-1)]*L$

$=Loc(a_{11})+11*L$

按列 $Loc(a_{ij})=Loc(a_{11})+[(j-1)*m+(i-1)]*L$

$=Loc(a_{11})+[(j-1)*6+(i-1)]*L$

$=Loc(a_{11})+(i+6j-7)*L$

因为按行存储与按列存储地址相同，得

$11=i+6j-7 \rightarrow i+6j=18 \rightarrow i=6,\ j=2$

## 二维数组的例题（2）

例:二维数组：0≤i≤5, 2≤j≤9；则按行存储$a_{24}$和按列存储$a_{05}$地址相同。

解：

$a_{02}$ $a_{03}$ $a_{04}$ $a_{05}$ .....$a_{09}$

$a_{12}$ $a_{13}$ $a_{14}$ $a_{15}$ .....$a_{19}$

$a_{22}$ $a_{23}$ $a_{24}$ $a_{25}$ .....$a_{29}$

..... ..... ..... ..... .....

$a_{52}$ $a_{53}$ $a_{54}$ $a_{55}$ .....$a_{59}$

6行

8列

按行 :$Loc(a_{24})=Loc(a_{02})+[i*n+j-2]*L= Loc(a_{02})+[2*8+2]*L$

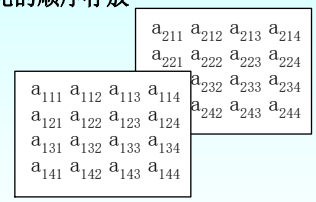
$=Loc(a_{02})+18L$

按列: $Loc(a_{ij})=Loc(a_{02})+[(j-2)*6+i]*L= Loc(a_{02})+[i+6j-12]*L$

得 $i+6j=30$ → $i=0, j=5$

## 2. 三维数组$A_{lmn}$的顺序存储结构

(1)按行优先的顺序存放




按行优先的顺序存放

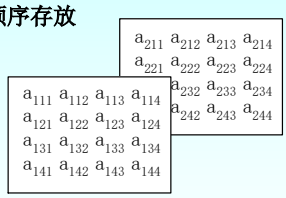
| $a_{111}$ | $a_{112}$ | $a_{113}$ | $a_{114}$ | $a_{121}$ | $a_{122}$ | … | $a_{144}$ | $a_{211}$ | $a_{212}$ | … | $a_{244}$ |
|---|---|---|---|---|---|---|---|---|---|---|---|

$Loc(a_{ijk})=Loc(a_{111})+[(i-1)*m*n+(j-1)*n+(k-1)]*L$

(1≤i≤l, 1≤j≤m, 1≤k≤n)

## 三维数组$A_{lmn}$的顺序存储结构（续）

(2)按列优先的顺序存放




按列优先的顺序存放

| $a_{111}$ | $a_{211}$ | $a_{121}$ | $a_{221}$ | $a_{131}$ | $a_{231}$ | $a_{141}$ | $a_{241}$ | … | … | $a_{144}$ | $a_{244}$ |
|---|---|---|---|---|---|---|---|---|---|---|---|

$Loc(a_{ijk})=Loc(a_{111})+[(k-1)*l*m+(j-1)*l+(i-1)]*L$

(1≤i≤l, 1≤j≤m, 1≤k≤n)

## 4.3 矩阵的压缩存储

4.3.1 特殊矩阵

4.3.2 稀疏矩阵

## 4.3.1 特殊矩阵

特殊矩阵：对非零元素按某种规律分布的矩阵。

（1）下三角矩阵

$$A_{nn}=\begin{bmatrix} a_{11} & 0 & \cdots & \cdots & 0 \\ a_{21} & a_{22} & 0 & \cdots & 0 \\ \vdots & \vdots & & & \vdots \\ a_{n1} & a_{n2} & a_{n3} & \cdots & a_{nn} \end{bmatrix}$$

特点:

当i<j时，$a_{ij}=0$

按“行优先顺序”存放非零元素，得到序列：

$\{a_{11}, a_{21}, a_{22}, a_{31}, a_{32}, a_{33}, \ldots, a_{n1}, a_{n2}, \ldots, a_{nn}\}$

共有1+2+3+…+(n-1)+n=n(n+1)/2个非零元素。

设每个元素占一个单元空间，非零元素$a_{ij}$的地址：

$Loc(a_{ij})=Loc(a_{11})+[i*(i-1)/2+(j-1)]*L$

$1≤j≤i≤n$

## 特殊矩阵的例题

例：在某下三角矩阵中，每个非零数据元素占1个存储单元。

已知$Loc(a_{11})=2000$，求$Loc(a_{43})$。

（1）$a_{11}$是第一个数据元素；

（2）$a_{00}$是第一个数据元素。

解：

（1）若从$a_{11}$开始计，则

$Loc(a_{43})=2000+3(1+3)/2+(3-1)= 2008$

（2）若从$a_{00}$开始计，则

$Loc(a_{11})= Loc(a_{00})+2=2000$→ $Loc(a_{00})=1998$

$Loc(a_{43})=1998+4(4+1)/2+3= 2011$

## （2）三对角矩阵

$$A_{nn}=\begin{bmatrix} a_{11} & a_{12} & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & 0 \\ a_{21} & a_{22} & a_{23} & 0 & \cdots & \cdots & \cdots & \cdots & 0 \\ 0 & a_{32} & a_{33} & a_{34} & 0 & \cdots & 0 & \cdots & 0 \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ 0 & \cdots & \cdots & \cdots & \cdots & 0 & a_{n-1,n-2} & a_{n-1,n-1} & a_{n-1,n} \\ 0 & \cdots & \cdots & \cdots & \cdots & \cdots & 0 & a_{n,n-1} & a_{nn} \end{bmatrix}$$

特点:当|i-j>1时，$a_{ij}=0$

按“行优先顺序”存放非零元素，得到序列：

$\{a_{11}, a_{12}, a_{21}, a_{22}, a_{23}, a_{32}, a_{33}, a_{34}, \ldots, a_{nn-1}, a_{nn}\}$

共有2+(n-2)*3+2=3n-2个非零元素。

非零元素$a_{ij}$的地址：

$Loc(a_{ij})=Loc(a_{11})+[2*(i-1)+(j-1)]*L$

$|i-j|\leqslant 1, 0\leqslant i\leqslant n, 0\leqslant j\leqslant n$

## 特殊矩阵的例题

例：对称矩阵$A_{nn}$（$a_{ij}=a_{ji}$）压缩存储，则只需要____个单元的存储空间；若按行优先将包括主对角线元素在内的下三角形的所有元素依次存放在一个一维数组B[k]中，1<i<n, 1<j<n, k=1,2,3……，则k和i、j的关系为：当i>=j时，k=_____；当i<j时，k=_____。

## 4.3.2 稀疏矩阵

o稀疏矩阵：矩阵中大多数元素为零，且非零元素的分布无一定规律。

o稀疏矩阵的三元组表示

$$A_{5*4}=\begin{pmatrix} 12 & 15 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 36 & 46 & 0 & 52 \\ 0 & 0 & 0 & 0 \\ 0 & 72 & 0 & 68 \end{pmatrix}$$

三元组表示

| row | col | e |
|---|---|---|
| 1 | 1 | 12 |
| 1 | 2 | 15 |
| 3 | 1 | 36 |
| 3 | 2 | 46 |
| 3 | 4 | 52 |
| 4 | 2 | 72 |
| 4 | 4 | 68 |

若row、col和e均占一个存储单元，且非零元素个数为tu，则需要3*tu个存储单元。

注：稀疏矩阵由非零元素的三元组及其行列数确定。

## 稀疏矩阵三元组表示的例题——转置

矩阵的转置——将一个矩阵由m×n的矩阵变换为n×m的矩阵，并且原矩阵的第i行变为新矩阵的第j列。

$$M=\begin{bmatrix} 0 & 12 & 9 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 3 & 0 & 0 & 0 & 0 & 14 \\ 0 & 0 & 24 & 0 & 0 & 0 \\ 0 & 18 & 0 & 15 & 0 & 0 \end{bmatrix} \qquad M^T=\begin{bmatrix} 0 & 0 & 3 & 0 & 0 \\ 12 & 0 & 0 & 0 & 18 \\ 9 & 0 & 0 & 24 & 0 \\ 0 & 0 & 0 & 0 & 15 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 14 & 0 & 0 \end{bmatrix}$$

M和$M^T$的区别：

（1） $M^T$将M矩阵的行值和列值互换；

（2）对于M中的每一个元素，它在$M^T$中行值和列值是互换的。

## 一般矩阵的转置算法

将mu行nu列的矩阵M转置为nu行mu列的矩阵T。

```
for (col=1; col<=nu;++col)
        for (row=1; row<=mu;++row)
                T[col][row] =M[row][col];
```

算法分析：时间复杂度为O(mu*nu)。

## 利用三元组表示的矩阵转置

$$M=\begin{bmatrix} 0 & 12 & 9 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 3 & 0 & 0 & 0 & 0 & 14 \\ 0 & 0 & 24 & 0 & 0 & 0 \\ 0 & 18 & 0 & 15 & 0 & 0 \end{bmatrix} \qquad M^T=\begin{bmatrix} 0 & 0 & 3 & 0 & 0 \\ 12 & 0 & 0 & 0 & 18 \\ 9 & 0 & 0 & 24 & 0 \\ 0 & 0 & 0 & 0 & 15 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 14 & 0 & 0 \end{bmatrix}$$

| row | col | e |
|---|---|---|
| 1 | 2 | 12 |
| 1 | 3 | 9 |
| 3 | 1 | 3 |
| 3 | 6 | 14 |
| 4 | 3 | 24 |
| 5 | 2 | 18 |
| 5 | 4 | 15 |

行值和列值互换

| row | col | e |
|---|---|---|
| 2 | 1 | 12 |
| 3 | 1 | 9 |
| 1 | 3 | 3 |
| 6 | 3 | 14 |
| 3 | 4 | 24 |
| 2 | 5 | 18 |
| 4 | 5 | 15 |

按行序重排

| row | col | e |
|---|---|---|
| 1 | 3 | 3 |
| 2 | 1 | 12 |
| 2 | 5 | 18 |
| 3 | 1 | 9 |
| 3 | 4 | 24 |
| 4 | 5 | 15 |
| 6 | 3 | 14 |

## 三元组表示的矩阵转置算法

三元组顺序表定义

```
#define MAXSIZE 12500
typedef struct {
    int i, j;    //非零元素的行下标和列下标
    ElemType e;
    }Triple;
typedef struct {
    Triple data[MAXSIZE+1];  //非零元素三元组,data[0]未用
    int mu,nu,tu;  //矩阵的行数、列数、非零个数
    }TSMatrix;
```

## 三元组表示的矩阵转置算法（续）

```
Status TransposeSMatrix(TSMatrix M , TSMatrix &T)
{ T.mu=M.nu; T.nu=M.mu; T.tu=M.tu;
  if (T.tu) {
    q=1;
    for (col =1; col <=M.nu;++col)
      for (p=1; p<=M.tu; ++p)
        if (M.data[p].j==col) {
          T.data[q].i=M.data[p].j;   T.data[q].j=M.data[p].i;
          T.data[q].e=M.data[p].e;   ++q;}
 return OK; }
```

算法分析：本算法主要工作是在p和col的两个循环中完成的，故算法的时间复杂度为O(nu*tu)，即与矩阵的列数和非0元素的个数的乘积成正比。

## 带行辅助向量的三元组 ——高效访问稀疏矩阵中的任一非零元素

三元组

| | row | col | e |
|---|---|---|---|
| 1 | 1 | 2 | 12 |
| 2 | 1 | 3 | 9 |
| 3 | 3 | 1 | 3 |
| 4 | 3 | 6 | 14 |
| 5 | 4 | 3 | 24 |
| 6 | 5 | 2 | 18 |
| 7 | 5 | 4 | 15 |

行辅助向量

| row | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| num[row] | 2 | 0 | 2 | 1 | 2 |
| cpot[row] | 1 | 3 | 3 | 5 | 6 |

- num[row]——第row行的非零元素个数
- cpot[row]——第row行的第一个非零元素在三元组中的位置

$$\begin{cases} \text{cpot}[1]=1 \\ \text{cpot}[\text{row}]=\text{cpot}[\text{row-1}]+\text{num}[\text{row-1}] & 2\le \text{row}\le \text{m} \end{cases}$$

## 带列辅助向量的三元组 ——提高转置的效率

三元组

| | row | col | e |
|---|---|---|---|
| 1 | 1 | 2 | 12 |
| 2 | 1 | 3 | 9 |
| 3 | 3 | 1 | 3 |
| 4 | 3 | 6 | 14 |
| 5 | 4 | 3 | 24 |
| 6 | 5 | 2 | 18 |
| 7 | 5 | 4 | 15 |

列辅助向量

| col | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| num[col] | 1 | 2 | 2 | 1 | 0 | 1 |
| cpot[col] | 1 | 2 | 4 | 6 | 7 | 7 |

转置后的三元组

| | row | col | e |
|---|---|---|---|
| 1 | 1 | 3 | 3 |
| 2 | 2 | 1 | 12 |
| 3 | 2 | 5 | 18 |
| 4 | 3 | 1 | 9 |
| 5 | 3 | 4 | 24 |
| 6 | 4 | 5 | 15 |
| 7 | 6 | 3 | 14 |

- num[col]——第col列的非零元素个数
- cpot[col]——第col列的第一个非零元素在转置后三元组中的位置

$$\begin{cases} \text{cpot}[1]=1 \\ \text{cpot}[\text{row}]=\text{cpot}[\text{col-1}]+\text{num}[\text{col-1}] & 2\le \text{col}\le \text{n} \end{cases}$$

## 快速转置的算法

算法思想：直接按照稀疏矩阵A的三元组表a.data的次序依次顺序转换，并将转换后的三元组放置于三元组表b.data的恰当位置。

前提：若能预先确定原矩阵A中每一列的(即B中每一行) 第一个非0元素在b.data中应有的位置，则在作转置时就可直接放在b.data中恰当的位置。因此，应先求得A中每一列的非0元素个数。

附设两个辅助向量num[ ]和cpot[ ]。

## 快速转置的算法（P100算法5.2）

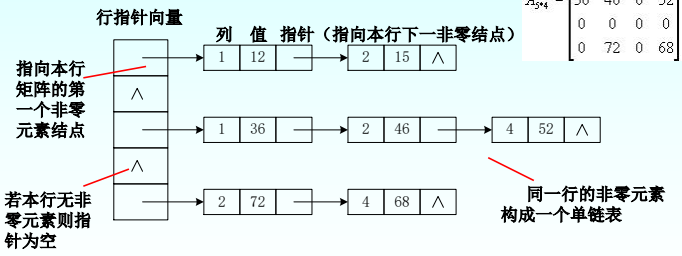
```
Status FastTransposeSMatrix(TSMatrix M, TSMatrix &T)
{  T.mu=M.nu; T.nu=M.mu; T.tu=M.tu;
   if (T.tu) {
     for (col =1; col <=M.nu;++col) num[col]=0;
     for (t =1; t<=M.tu;++t) ++num[M.data[t].j];
     cpot[1]=1;
     for (col =2; col <=M.nu;++col)
       cpot[col]=cpot[col-1]+num[col-1];
    for (p=1; p<=M.tu; ++p) {
       col=M.data[p].j;   q=cpot[col];
       T.data[q].i=M.data[p].j;  T.data[q].j=M.data[p].i;
       T.data[q].e=M.data[p].e;  ++cpot[col];}
   }return OK;
}
```

算法分析：本算法中有4个并列的单循环，循环次数分别是nu和tu，因而总的时间复杂O(nu)+O(tu)=max(O(nu),O(tu)) 。

## 4. 4数组的链式存储结构

### 1. 带行指针向量的单链表

$$A_{5*4}=\begin{bmatrix}12&15&0&0\\0&0&0&0\\36&46&0&52\\0&0&0&0\\0&72&0&68\end{bmatrix}$$




若矩阵行数为m，非零元素个数为tu，则所用存储单元为3tu＋m；

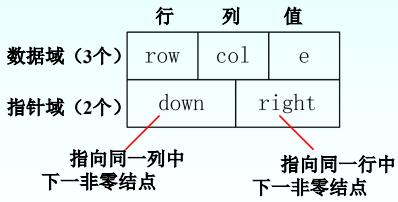
若每行中非零元素个数不超过d，则存取元素的时间复杂度为O（d）。

思考题：如何构造带列指针向量的单链表？

## 2. 十字链表结构

十字链表结构是一种动态存储结构。

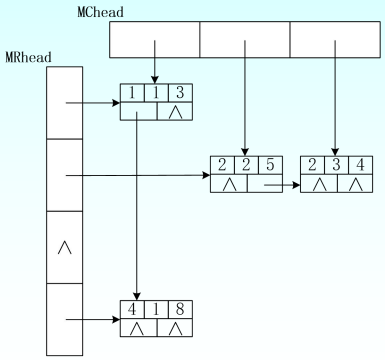
在十字链表中，每个非零元素用如下结构表示:




## 十字链表的例题

例：

$$A_{4*3}=\begin{pmatrix}3&0&0\\0&5&4\\0&0&0\\8&0&0\end{pmatrix}$$




实质：带行指针向量的单链表和带列指针向量的单链表构成

## 循环十字链表的例题

例：

$$A_{4*3}=\begin{pmatrix}3&0&0\\0&5&4\\0&0&0\\8&0&0\end{pmatrix}$$

